

LLM-Powered Query Monitoring and Optimization Using Reproducible External Data Workloads

- Abstract
- Introduction
- System Design
- Implementation
 - 4.1 Query History Schema
 - 4.2 LLM Analysis Pipeline
- Experimental Setup
- Results
- Discussion
- Limitations
- Conclusion
- References
- Appendix A: Dataset Manifest
- Appendix B: Full Query History Schema

LLM-Powered Query Monitoring and Optimization Using Reproducible External Data Workloads

Abstract

Manual SQL review does not scale in modern data warehouses. We present an LLM-powered query monitoring framework that identifies inefficient or risky SQL queries, recommends optimized rewrites, and validates recommendations through automated correctness checks. The system ingests public datasets, executes a controlled query workload against DuckDB, stores execution metadata in a SQLite query history database, analyzes queries with an LLM, and validates rewrites through semantic comparison. In our evaluation across three public datasets and eight queries (four baseline and four intentionally inefficient variants), the framework achieved **100% detection accuracy** for common SQL anti-patterns with **zero false positives**, a **75% semantic match rate** across optimized rewrites, and a total LLM API cost of **\$0.000671**. The evaluation was conducted on a pilot-scale workload that used a 3-row version of each dataset, enabling sub-millisecond runtimes. These results should be interpreted as pilot-scale evidence of a reproducible query guardrail, not as a production-scale benchmark.

Introduction

Data warehouses routinely process large volumes of analytical SQL, yet inefficient query patterns are often discovered only after they cause slow runtimes, unnecessary scans, or high cloud costs. Manual SQL review is valuable but does not scale across ad hoc workloads, notebooks, dashboards, and scheduled pipelines. This paper presents an **open-source, reproducible framework** for evaluating whether large language models can serve as a lightweight query-monitoring guardrail.

The framework is implemented as a public artifact at <https://github.com/shouvik-sharma/query-monitoring-framework>. It downloads public datasets, creates a local DuckDB workload, records executions in a SQLite query-history store, sends query text and metadata to an LLM analyzer, stores recommendations, executes rewritten queries, and reports correctness and cost metrics. The goal is practical reproducibility: a reader should be able to clone the repository, run the scripts, and inspect the same schema, workloads, and analysis reports.

System Design

The architecture is organized around six implementation components. Each component maps directly to one or more scripts in the repository.

```
DataSource -> ExecutionEngine -> QueryHistoryStore
              -> LLMAnalyzer -> RecommendationEngine
              -> ReportGenerator
```

Component	Script(s)	Responsibility
DataSource	<code>maintain_datasets.py</code>	Downloads, validates, and stages public datasets.
ExecutionEngine	<code>execute_query_workload.py</code>	Runs baseline and intentionally inefficient SQL against DuckDB.
QueryHistoryStore	<code>create_db.py</code> , <code>schema/query_history_schema.sql</code>	Creates and maintains the SQLite query-history database.
LLMAnalyzer	<code>llm_analysis.py</code>	Sends query context to the LLM and records scores, issues, latency, token counts, and cost.
RecommendationEngine	implemented inside <code>llm_analysis.py</code>	Extracts and stores rewritten SQL recommendations returned by the LLM.
ReportGenerator	<code>generate_report.py</code> , <code>cost_analysis_report.py</code>	Computes evaluation summaries, semantic-match checks, and cost reports.

This decomposition separates data preparation, execution, analysis, and reporting so that each step can be inspected and re-run independently.

Implementation

4.1 Query History Schema

The SQLite database schema is defined in `schema/query_history_schema.sql`. Section 4.1 summarizes the schema for readability; the full `CREATE TABLE` script is provided in Appendix B.

Table	Key Columns	Purpose
workloads	id, name, description, engine, created_at	Groups queries for an experiment run.
datasets	id, name, source_url, local_path, description, ingested_at	Tracks public dataset sources and local paths.
queries	id, workload_id, dataset_id, query_text, query_label, inefficiency_type, expected_issue, is_baseline, created_at	Stores baseline and intentionally inefficient SQL queries.
query_executions	id, query_id, execution_label, engine, status, runtime_ms, rows_returned, result_checksum, sample_output, explain_plan, estimated_cost, error_message, executed_at	Captures execution outcomes for original and rewritten queries.
llm_analyses	id, query_id, model, prompt_version, score, score_reason, issues_found, input_tokens, output_tokens, llm_cost_usd, latency_ms, error_message, analyzed_at	Stores LLM scores, explanations, token counts, cost, and latency.
recommendations	id, query_id, llm_analysis_id, recommended_query, improvement_reason, expected_improvement_category, improvement_suggestion, created_at	Stores LLM-generated SQL rewrites and rationale.
cost_comparisons	id, query_id, recommendation_id, original_execution_id, rewritten_execution_id, original_runtime_ms, rewritten_runtime_ms, runtime_improvement_pct, original_rows, rewritten_rows, rows_match, checksum_match, semantic_match, validation_status, llm_total_cost_usd, net_cost_improvement, notes, created_at	Records before/after validation and aggregate cost-comparison metadata.

The schema is intentionally simple: every recommendation remains linked to the original query, the LLM analysis that produced it, and the execution records used to validate semantic equivalence.

4.2 LLM Analysis Pipeline

The LLM analysis pipeline is implemented in `llm_analysis.py`. For each stored query, the script builds a prompt containing query text, expected issue labels, and execution context, then requests a structured JSON response from the configured OpenAI model. The response is parsed into an analysis score, issue list, explanation, and optional rewritten SQL recommendation.

Recommendations are not accepted purely on model output. The rewritten SQL is executed through the same DuckDB workload harness, and the resulting row counts and checksums are compared against the original query where possible. This validation step is what allows the framework to distinguish useful rewrites from syntactically plausible but semantically incorrect recommendations.

Experimental Setup

The experiment uses three public datasets: USGS earthquake records, NOAA weather records, and the AWID intrusion-detection dataset. For the pilot evaluation, each table is reduced to three rows to keep the run deterministic and inexpensive. The workload contains eight queries: four baseline queries and four intentionally inefficient variants designed to expose common SQL anti-patterns such as unnecessary cross joins and avoidable broad scans.

The reproducible pipeline is script-driven. `maintain_datasets.py` prepares data, `create_db.py` initializes the SQLite store from `schema/query_history_schema.sql`, `execute_query_workload.py` executes the SQL workload, `llm_analysis.py` performs LLM scoring and rewrite generation, and the report scripts summarize accuracy, semantic match, runtime, and cost.

Results

The pilot run achieved 100% detection accuracy across the intentionally inefficient queries and produced zero false positives for the baseline queries. Across optimized rewrites, 75% of recommendations matched the original query semantics under the implemented validation checks. The total LLM API cost for the evaluated workload was \$0.000671.

These results are intentionally reported at pilot scale. Because the tables contain only three rows each, runtime differences are not meaningful production-performance claims. The stronger result is that the framework can identify known anti-patterns, produce structured recommendations, validate rewrites, and report cost with a fully reproducible local workflow.

Discussion

The evaluation suggests that LLMs can be useful as query-review assistants when their outputs are constrained by a reproducible execution and validation loop. The framework does not assume that a model recommendation is correct; it records the recommendation, executes it, and checks whether the rewritten query preserves expected semantics. This design is especially important for SQL optimization, where an apparently cleaner query may silently change results.

The current system is best viewed as an artifact for controlled experimentation and practitioner prototyping. Its value lies in making each step auditable: dataset preparation, workload construction, LLM analysis, rewrite validation, and cost reporting are all implemented as separate scripts with persistent records in the query-history database.

Limitations

- **Pilot-scale evaluation:** The experiment used only 3 rows per table to isolate detection behavior from volume effects; runtime deltas are dominated by measurement noise.
- **Small query count:** Only 8 queries were evaluated (4 baseline and 4 intentionally inefficient). Larger query corpora are required for statistical confidence.
- **DuckDB-only runtime:** The execution engine currently targets DuckDB and does not yet support production systems such as Snowflake, BigQuery, Redshift, or PostgreSQL.
- **SQLite query history:** SQLite is appropriate for the local artifact but is not designed for high-concurrency production telemetry.
- **LLM dependency:** Cost, latency, and recommendation quality depend on the configured OpenAI model. Offline simulations use deterministic placeholders and should not be interpreted as live-model results.
- **Limited rewrite validation:** The framework checks row counts and checksums where applicable, but it does not prove semantic equivalence for all SQL constructs.

Conclusion

This paper presents an open-source framework for evaluating LLM-powered SQL query monitoring and optimization. The framework links public datasets, a reproducible DuckDB workload, a SQLite query-history schema, structured LLM analysis, rewrite validation, and cost reporting into a single artifact. In a pilot workload of eight queries across three public datasets, it achieved 100% inefficient-query detection, zero false positives, a 75% semantic-match rate for rewrites, and \$0.000671 in total LLM API cost.

The results do not establish production-scale performance. Instead, they demonstrate that LLM query guardrails can be evaluated transparently, cheaply, and reproducibly. Future work should expand the workload size, add production warehouse connectors, test larger datasets, and evaluate multiple model families under the same validation harness.

References

1. Framework repository. <https://github.com/shouvik-sharma/query-monitoring-framework>
2. USGS Earthquake Hazards Program. Earthquake Catalog. <https://www.usgs.gov/programs/earthquake-hazards>
3. NOAA National Centers for Environmental Information. Global Hourly Data. <https://www.ncei.noaa.gov/>
4. AWID Intrusion Detection Dataset. Kaggle Mirror. <https://github.com/krishanuskr/AWID-Intrusion-Detection-System-2020>
5. DuckDB. Fast analytical database. <https://duckdb.org/>
6. OpenAI. GPT-4o-mini. <https://platform.openai.com/docs/models/gpt-4o-mini>

Appendix A: Dataset Manifest

The artifact uses public external datasets from USGS, NOAA, and AWID. The repository scripts download and stage these datasets locally so that the workload can be recreated without private or proprietary data.

Appendix B: Full Query History Schema

The full schema below is copied from `schema/query_history_schema.sql`.

```
-- query_history.db schema
-- LLM-Powered Query Monitoring Framework
-- SQLite-compatible

-- Workload definitions (groups of queries for an experiment run)
CREATE TABLE IF NOT EXISTS workloads (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    description TEXT,
    engine TEXT NOT NULL DEFAULT 'duckdb',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Dataset sources
CREATE TABLE IF NOT EXISTS datasets (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    source_url TEXT,
    local_path TEXT,
    description TEXT,
    ingested_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- The queries themselves (baselines and intentional variants)
CREATE TABLE IF NOT EXISTS queries (
```

```

        id INTEGER PRIMARY KEY,
        workload_id INTEGER REFERENCES workloads(id),
        dataset_id INTEGER REFERENCES datasets(id),
        query_text TEXT NOT NULL,
        query_label TEXT,
        inefficiency_type TEXT,
        expected_issue TEXT,
        is_baseline BOOLEAN DEFAULT 1,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );

-- Query executions (original and rewritten)
CREATE TABLE IF NOT EXISTS query_executions (
    id INTEGER PRIMARY KEY,
    query_id INTEGER NOT NULL REFERENCES queries(id),
    execution_label TEXT NOT NULL CHECK (execution_label IN ('original', '
engine TEXT NOT NULL,
status TEXT NOT NULL CHECK (status IN ('success', 'error', 'timeout'))
runtime_ms INTEGER,
rows_returned INTEGER,
result_checksum TEXT,
sample_output TEXT,
explain_plan TEXT,
estimated_cost REAL,
error_message TEXT,
executed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- LLM analysis results
CREATE TABLE IF NOT EXISTS llm_analyses (
    id INTEGER PRIMARY KEY,
    query_id INTEGER NOT NULL REFERENCES queries(id),
    model TEXT NOT NULL,
    prompt_version TEXT,
    score INTEGER CHECK (score >= 0 AND score <= 100),
    score_reason TEXT,
    issues_found TEXT,
    input_tokens INTEGER,
    output_tokens INTEGER,
    llm_cost_usd REAL,
    latency_ms INTEGER,
    error_message TEXT,
    analyzed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- LLM-generated recommendations
CREATE TABLE IF NOT EXISTS recommendations (
    id INTEGER PRIMARY KEY,
    query_id INTEGER NOT NULL REFERENCES queries(id),
    llm_analysis_id INTEGER NOT NULL REFERENCES llm_analyses(id),
    recommended_query TEXT NOT NULL,
    improvement_reason TEXT,
    expected_improvement_category TEXT,
    improvement_suggestion TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Before/after cost comparisons
CREATE TABLE IF NOT EXISTS cost_comparisons (
    id INTEGER PRIMARY KEY,
    query_id INTEGER NOT NULL REFERENCES queries(id),
    recommendation_id INTEGER NOT NULL REFERENCES recommendations(id),
    original_execution_id INTEGER REFERENCES query_executions(id),
    rewritten_execution_id INTEGER REFERENCES query_executions(id),
    original_runtime_ms INTEGER,
    rewritten_runtime_ms INTEGER,
    runtime_improvement_pct REAL,
    original_rows INTEGER,
    rewritten_rows INTEGER,
    rows_match BOOLEAN
);

```

```
rows_match BOOLEAN,  
checksum_match BOOLEAN,  
semantic_match BOOLEAN,  
validation_status TEXT CHECK (validation_status IN ('match', 'mismatch'  
llm_total_cost_usd REAL,  
net_cost_improvement TEXT,  
notes TEXT,  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

